Universidad del Valle de Guatemala

Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3067 Deep Learning, Semestre II 2026



DELVALLE
GRUPO EDUCATIVO

Laboratorio 9: Series de tiempo y forecasting

Roberto Barreda #23354
Mia Fuentes #23775

Guatemala, 28 de octubre de 2026

1. Investigación: herramientas para forecasting en PyTorch

Tabla 1. Clases y funciones usadas para construir, entrenar y evaluar los modelos.

Herramienta	Propósito y parámetros relevantes
nn.LSTM y nn.GRU	Recorren la secuencia con un estado oculto. input_size es el número de variables por paso, hidden_size el tamaño del estado, num_layers las capas apiladas, batch_first=True usa (batch, L, features) y dropout se aplica entre capas. output tiene forma (batch, L, hidden) con la última capa en cada paso, h_n (num_layers, batch, hidden) con el último estado de cada capa y c_n, solo en LSTM, el estado de celda con la misma forma.
nn.Conv1d causal y dilatada	Conv1d(in_channels, out_channels, kernel_size, dilation). El padding causal agrega (k - 1)d ceros solo a la izquierda, así la salida en t solo ve t y el pasado. En una TCN con bloques de dos convoluciones y d = 2^i el campo receptivo es R = 1 + 2(k - 1)(2^n - 1). Con k = 3 y n = 6 bloques R = 253 horas, mayor que L = 168.
nn.TransformerEncoderLayer	Atención multicabeza más una red feed-forward con residuales. d_model es la dimensión de cada paso, nhead el número de cabezas (divide a d_model) y dim_feedforward la capa oculta de la red feed-forward. La atención no conoce el orden, por eso se suma una codificación posicional sinusoidal o aprendida de forma (L, d_model) a la entrada proyectada.
NeuralForecast y StatsForecast	Reciben un DataFrame largo con unique_id (serie), ds (marca de tiempo) e y (valor), más covariables como columnas. cross_validation(df, h, n_windows, step_size, refit) genera n_windows orígenes separados por step_size pasos; refit=False entrena una vez y refit=k reentrena cada k ventanas. Para este protocolo h = 24, n_windows = 92 y step_size = 24.
BaseChronosPipeline.predict_quantiles	Pronóstico zero-shot con Chronos-Bolt. La entrada es un tensor (batch, T) o una lista de tensores 1D con la historia de cada serie, prediction_length es el horizonte (24) y quantile_levels los cuantiles (0.1, 0.5, 0.9). Devuelve los cuantiles (batch, H, Q) y la media (batch, H).

Un modelo local se entrena por serie y captura sus particularidades, pero no comparte información y escala mal con cientos de clientes. Un modelo global usa todas las series normalizadas, aprende patrones comunes y sirve para series con poca historia. Teacher forcing entrega al decoder el valor real del paso anterior durante el entrenamiento; estabiliza el aprendizaje, pero en inferencia el modelo recibe sus propias predicciones y los errores se acumulan (exposure bias). RevIN normaliza cada ventana con su media y desviación, el modelo predice en esa escala y la salida se desnormaliza con los mismos estadísticos; ante cambios de nivel el modelo aprende la forma de la curva y no su nivel. La pinball loss para el cuantil q es max(q(y - p), (q - 1)(y - p)), con p la predicción, y su mínimo esperado es el cuantil q de la distribución, por eso entrenar con q = 0.1 y 0.9 produce un intervalo que debe cubrir 80 % de los valores. MASE divide el MAE de cada serie entre el MAE del seasonal naive en entrenamiento y da el mismo peso a cada cliente; WAPE divide la suma de errores absolutos entre el consumo total del panel y lo dominan los clientes grandes. Ninguna se indefine con consumos cero, a diferencia del MAPE.

2. Exploración y preparación de la serie

Cada registro es la potencia media en kW de 15 minutos, la energía horaria en kWh es el promedio de los 4 registros. Las series se normalizan con z-score por cliente usando media y desviación de entrenamiento y las métricas se calculan en kWh.

¿Cuántos clientes quedan después del filtro del periodo 2012 a 2014? ¿Cuántas observaciones horarias tiene cada serie y cuántas el panel?

Quedan 317 de 370 clientes. Se excluyen 50 que inician después del 2012-01-01, 2 que dejan de consumir antes del cierre y MT_131, con 1,230 horas seguidas en cero. Cada serie tiene 26,304 observaciones y el panel 8,338,368.

¿Cómo tratarán los dos días de cambio de horario de cada año?

En marzo la hora 01:00 viene en cero y se reemplaza por el promedio de las 00:00 y las 02:00. En octubre esa hora acumula dos horas y se divide entre 2. Cada día conserva 24 horas sin huecos ni picos falsos.

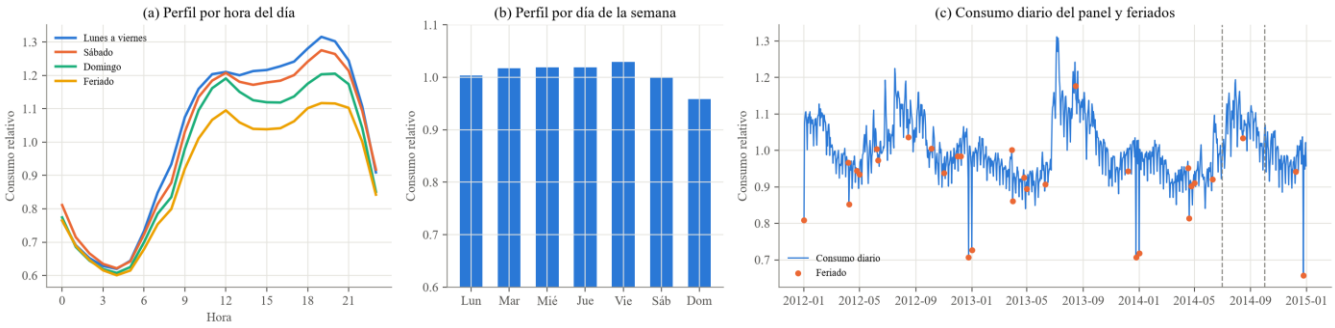


Figura 1. Perfil por hora y por día de la semana y consumo diario del panel con feriados de Portugal. El consumo de cada cliente se divide entre su media de entrenamiento; las líneas punteadas marcan el inicio de validación y prueba.

¿Cómo cambia el consumo en Navidad y Año Nuevo?

El 25 de diciembre cae a 67 % a 70 % del mismo día de la semana en las tres semanas previas y el 1 de enero a 78 % a 79 %. El 24 baja entre 3 % y 9 % y el 31 entre 2 % y 4 %, con la caída en la tarde y noche; el 26 vuelve al nivel normal. Los feriados reducen el pico de mediodía y tarde entre 10 % y 15 % frente a un día laboral.

¿Qué tan distintas son las escalas entre clientes? ¿Qué normalización proponen?

El consumo medio va de 4.33 a 48,818 kWh por hora con mediana de 148.13 kWh, una razón de 11,268. Con una normalización global los clientes pequeños quedan cerca de cero y la pérdida la dominan los grandes, por eso se usa z-score por serie con estadísticos de entrenamiento. RevIN por ventana queda como opción de los modelos.

¿Cómo codifican las covariables de calendario conocidas a futuro?

Hora, día de la semana y mes con seno y coseno, 6 columnas que conservan la continuidad cíclica en lugar de 43 columnas one-hot. El feriado es binario según el calendario oficial de Portugal, que suspendió 4 feriados entre 2013 y 2015. Todas dependen solo de la marca de tiempo y se conocen para las 24 horas futuras.

La clase VentanasDataset genera x de forma (batch, 168, 8) con el consumo normalizado y el calendario pasado, xf de forma (batch, 24, 7) con el calendario futuro e y de forma (batch, 24). Una ventana pertenece a un bloque si sus 24 horas objetivo caen en él; la entrada puede tomar horas del bloque anterior porque ya son conocidas.

Tabla 2. Ventanas generadas por el panel de 317 clientes con L = 168, H = 24 y stride de 1 hora.

Bloque	Horas objetivo	Ventanas por serie	Ventanas del panel
Entrenamiento	2012-01-08 a 2014-06-30	21,697	6,877,949
Validación	2014-07-01 a 2014-09-30	2,185	692,645
Prueba	2014-10-01 a 2014-12-31	2,185	692,645
Prueba, origen diario 00:00	2014-10-01 a 2014-12-31	92	29,164

3. Auditoría de fugas

Tabla 3. Fugas vistas en clase y la forma en que el pipeline las evita y las verifica.

Fuga	**Cómo se evita**	**Verificación en el notebook**
Split aleatorio de ventanas	Los bloques se definen por fecha de las horas objetivo; solo se baraja dentro de entrenamiento.	assert de rangos de fechas disjuntos y ordenados.
Escalar con toda la serie	Media y desviación por cliente con datos hasta 2014-06-30.	Multiplicar validación y prueba por 10 no cambia los estadísticos.
Early stopping con la prueba	entrenar() solo recibe entrenamiento y validación. backtest registra cada modelo y bloquea una segunda evaluación en prueba.	assert dentro de backtest.
Covariables mal alineadas	x termina a las 23:00 previas al origen y xf es el calendario de las horas objetivo; el feriado sale del calendario oficial.	assert de alineación y cambiar valores desde el origen no altera x.

4. Protocolo de validación y métricas

La función backtest(modelo) recorre los 92 orígenes de prueba; en cada uno el modelo recibe el panel en kWh hasta las 23:00 del día anterior y devuelve las 24 horas siguientes. Los pesos se entrenan una vez con datos hasta el 2014-09-30 y, con reentrenar=True, el modelo se ajusta al inicio de cada mes con ventana expandible. Se reportan MAE, RMSE, WAPE, MAPE y MASE en kWh sobre todos los clientes y orígenes, además del error por paso del horizonte, por hora del día, por cliente y en la semana del 22 al 28 de diciembre. La escala de MASE es el MAE del seasonal naive de 24 h en entrenamiento de cada cliente. En validación el seasonal naive de 24 h obtiene MAE de 52.42 kWh, WAPE de 0.069 y MASE de 1.00, frente a 69.35 kWh y MASE de 1.20 del de 168 h.

¿Qué le pasa al MAPE con clientes de consumo cercano a cero?

Hay 3,769 horas con consumo cero en validación donde el MAPE no está definido. En horas con menos de 1 kWh el MAPE del seasonal naive llega a 351 % contra 13.4 % en el resto, y los clientes de menor consumo tienen el MAPE más alto (Spearman de -0.41). Por eso la comparación se basa en WAPE y MASE.

Repositorio: https://github.com/bar23354/DL-Lab09